



Eindhoven University of Technology
Department of Mathematics and Computer Science
Software Engineering and Technology Research Group

Configurable Monitoring Infrastructure for ROS 2 Applications

Master's Thesis

Shikai Jin
s.jin1@student.tue.nl

Supervisor:
Dr. Ivan Kurtev

Eindhoven, June 2026

Abstract

Robot Operating System 2 (ROS 2) is a middleware that is widely used to build robotic applications. A ROS 2 application often contains many nodes that communicate through topics, services, and actions. This makes the application flexible, but it can also make its runtime behaviour difficult to understand, so the running system has to be monitored. Existing monitoring solutions often focus on one communication type, one monitoring method, or one deployment setup. Developers therefore need to write extra monitoring code for each new application.

In this thesis, we present a configurable monitoring infrastructure for ROS 2 applications. We built it as a reusable foundation for collecting, processing, routing, checking, and exporting runtime data from heterogeneous ROS 2 interfaces. The infrastructure observes topics, service events, and action feedback and status, and converts these different observations into a common data format. Each data record includes the observed data and useful context, such as its source, time, session, and identifier.

Users describe the monitoring setup in YAML files. They can select data sources, reduce or filter data, choose outputs, and connect property-specific checking components. New data converters, verdict services, input sources, and output exporters can also be added as separately developed components without changing the monitoring core. The deployment configuration are generated automatically from the user's selections.

The infrastructure runs in two ways. In the integrated setup, data collection and checking run together. In the split setup, data is collected near the robot and sent to a separate verifier runs on a different machine. The concrete prototype supports file and MQTT transport and provides processing steps such as field selection, rate limiting, and change filtering.

We evaluated the prototype through case studies covering topic, service, and action monitoring, including a Navigation 2 case study, complemented by unit tests, Docker Compose deployments, offline replay, and a multi-robot scenario. Across these scenarios, the same data format and processing chain connected ROS 2 topics, service events, and action progress to reusable checking components in both integrated and split deployments.

Contents

Contents	2
1 Introduction	4
1.1 Context and Motivation	4
1.2 Problem Statement	5
1.3 Research Objective and Questions	5
1.4 Approach	6
1.5 Contributions	6
1.6 Thesis Outline	7
2 Background	8
2.1 ROS 2 Applications	8
2.1.1 Nodes and the ROS Graph	8
2.1.2 Topics	8
2.1.3 Services	8
2.1.4 Actions	10
2.1.5 Discovery, Middleware, and Quality of Service	10
2.2 Runtime Monitoring and Runtime Verification	11
2.2.1 Main Terms	11
2.2.2 Online and Offline Monitoring	11
2.2.3 Passive and Active Monitoring	12
2.2.4 Centralized and Distributed Monitoring	12
2.3 Observing ROS 2 Applications	12
2.3.1 Observing Topics	12
2.3.2 Observing Services	12
2.3.3 Observing Actions	13
2.3.4 Types and Discovery for Monitors	13
2.4 Main Monitoring Tasks	13
2.4.1 Collecting Data	13
2.4.2 Processing Data	14
2.4.3 Routing and Storage	14
2.4.4 Checking and Verdicts	14
2.4.5 Using the Results	14
2.5 Variability and Feature Models	14
2.6 Deployment and Transport	15
2.6.1 Monitoring Organisations	15
2.6.2 Transport Protocols	18

3	Related Work	19
3.1	How the Approaches Are Compared	19
3.2	ROSMonitoring	19
3.3	ROMoSu	20
3.4	Generated Monitors for ROS 2	20
3.5	RTAMT	20
3.6	Tracing and Performance Monitoring	20
3.7	Network-Level Monitoring and Security	21
3.8	Containerized and Distributed Verification	21
3.9	Digital-Twin Runtime Verification	21
3.10	Field-Based Testing Guidance	21
3.11	Summary Comparison	22
3.12	Capability Matrix	23
3.13	Research Gap	24
4	Design	25
4.1	Feature Model as Design Input	25
4.2	Selected Feature Coverage	26
4.3	Design Requirements	27
4.4	Infrastructure Overview	27
4.5	Pipeline Roles	28
4.6	Common Data Record	29
4.7	Deployment Topologies	30
5	Implementation	32
5.1	Technology Choices and Code Organization	32
5.2	The Monitor Process	33
5.3	Collecting ROS 2 Observations	33
5.4	Records and Transformers	34
5.5	Exporters, Sources, and Transports	35
5.6	Converters and Verdict Services	35
5.7	Configuration and Graph Wiring	36
5.8	The Verifier Runtime	37
5.9	Generating Deployment Configurations	37
6	Evaluation	41
7	Discussion	42
8	Conclusions	43
	Bibliography	44
	Appendix	46
A	Appendix	47

Chapter 1

Introduction

1.1 | Context and Motivation

Robots often work in changing environments. Their behaviour depends on software, hardware, sensor data, communication, and the physical world. Some events can only be seen while the robot is running. For this reason, testing a robot before deployment is important, but it is not always enough. Developers also need to observe the robot during operation.

Many modern robots are built with the Robot Operating System 2 (ROS 2). Despite its name, it is not an operating system in the traditional sense, but run on top of a general-purpose operating system such as Linux. It provides a set of software libraries and tools for assembling and building functional components of a robotic system, and built-in packages for components working together to create different robot applications [1].

A ROS 2 application is composed of *nodes*: programs with a focused task, such as reading a sensor or planning a route. Nodes communicate through typed interfaces. *Topics* are mainly used for continuous data streams, *services* for request-response interactions, and *actions* for long-running tasks that provide feedback while they run [1, 2]. Chapter 2 explains these mechanisms in detail.

A common way to observe a system during operation is *runtime monitoring*. Runtime monitoring collects information from a running system. The collected information can be used to find errors, understand runtime behaviour, or check whether the system follows expected rules [3, 4]. The component that performs this task is called a *monitor*: it observes the running system, records what it sees as observations, and can analyse these observations, for example by checking them against an expected property and reporting a verdict. Unlike model checking, runtime monitoring does not study every possible system state. It studies the behaviour that actually happens during an execution. When the emphasis is on checking the observed behaviour against a specification, the technique is also called runtime verification [4].

Runtime monitoring is particularly relevant for ROS 2 applications. The modular node structure makes it easier to reuse and replace software parts, but it also spreads the behaviour of the robot across many nodes and interfaces. Important information may come from several topics, services, or actions. It can therefore be difficult to understand what the full system is doing by looking at any single component.

ROS 2 already provides several ways to observe a system. For example, a monitor can subscribe to a topic, read service introspection events, or observe the hidden topics used by actions. Lower-level tools can also trace ROS 2 execution. The difficult part is turning these possibilities into a reusable monitoring setup: developers still need to decide what to observe, how much data to keep, where to send the data, and how to check it. These decisions are often implemented again for every application. This thesis presents a configurable monitoring infrastructure that turns these separate observation possibilities into a reusable monitoring

foundation for ROS 2 applications.

1.2 | Problem Statement

ROS 2 applications need runtime monitoring. A robotic application interacts with an environment that testing cannot fully cover: failures caused by timing, sensor noise, hardware behaviour, or unexpected environment states often appear only while the system is running. In addition, because a ROS 2 application is distributed over many nodes, no single component has a complete view of the system behaviour. Runtime monitoring is therefore needed to detect errors, to understand what the system is doing, and to check whether the running system still satisfies its requirements.

Setting up monitoring for a concrete application, however, involves many configuration options and scenarios. The developer has to decide *what* to observe: which topics, services, and actions carry the relevant behaviour. They have to decide *how* to observe it: through extra subscriptions, service introspection, lower-level tracing, or instrumented application code. They have to decide how much data to keep: complete messages support offline replay, while a few fields from a high-rate stream may be all that a check needs. They have to decide how the properties of interest are specified and checked: as simple thresholds, state machines, temporal-logic formulas, or project-specific checking services. Finally, they have to decide *where* the monitor is deployed: together with the application, close to the robot, or on a separate machine that receives the observations over a network.

All of these choices have advantages and disadvantages, and they interact with each other. For example, observing everything makes checking flexible but can overload storage and transport, while a remote verifier eases the load on the robot but adds a network boundary. Because the choices interact, an ad-hoc combination assembled for one application rarely carries over to the next one. There is a need for a systematic way to specify a monitoring solution: a description of the available options and a means to select among them.

The second problem is that implementing a chosen monitoring solution is time consuming. Much of the required code, such as subscribing to interfaces, decoding typed messages, attaching timestamps and context, serializing data, and transporting it to checkers and files, is essentially the same in every project, yet it is written again for each application. This repeated implementation effort should be reduced by automation and by reusable, configurable infrastructure support.

These two problems, the systematic specification of a monitoring solution and the reusable implementation of it, are the subject of this thesis. The thesis is not concerned with how properties are specified or how they are checked. Existing specification languages and checking tools can be attached to the infrastructure through defined interfaces; the infrastructure delivers the observations they need.

1.3 | Research Objective and Questions

The objective of this thesis is to design, implement, and evaluate a configurable monitoring infrastructure for ROS 2 applications. The infrastructure should collect different ROS 2 observations in a common way. It should also allow users to select the observation, processing, checking, output, and deployment setup through configuration rather than through new monitoring code.

The following research questions guide the work:

RQ1 What monitoring capabilities currently exist for ROS 2, and which observation gaps remain?

- RQ2** What are the main commonalities and variabilities among existing monitoring architectures for ROS 2 applications?
- RQ3** What does a generic reference architecture for ROS 2 monitoring look like, and can it incorporate the features identified in RQ2?
- RQ4** To what extent does the implemented infrastructure support the monitoring needs of realistic ROS 2 applications?

1.4 | Approach

Each research question is answered as follows.

To answer RQ1, we review existing research and tools for monitoring ROS and ROS 2 applications. We group the approaches by the level at which they observe the system: ROS graph instrumentation, network-level observation, and lower-level tracing. The review identifies which ROS 2 elements each approach can observe and which observation gaps remain (Chapters 2 and 3).

To answer RQ2, we perform a domain analysis of the reviewed monitoring approaches. We identify the tasks they share, such as collecting, processing, routing, checking, and reporting, and the points where they differ, such as the observable elements, the monitoring architecture, the monitoring mode, the system configuration, and the implementation technique. We capture these commonalities and variabilities in a feature model (Chapter 4).

To answer RQ3, we design a reference architecture for ROS 2 monitoring and implement it in a Python prototype (Chapters 4 and 5). The architecture is a configurable pipeline. A collector observes a configured topic, service event stream, or action feedback and status stream and stores each observation in a common data record that carries the payload together with source information, timestamps, a session identifier, and a record identifier. Optional transformers reduce or reshape records, dispatchers route them to outputs and checking components, converters prepare them for property-specific verdict services, and exporters deliver data and verdicts to files, standard output, or MQTT. The same pipeline supports an integrated deployment, in which all steps run in one ROS 2-facing process, and a split deployment, in which a robot-side monitor sends records over MQTT to a separate verifier without a ROS 2 dependency. We then check the architecture against the feature model from RQ2 to determine which of the identified features it can incorporate.

To answer RQ4, we evaluate the prototype through case studies that cover the three ROS 2 communication mechanisms and both deployment forms, including a Navigation 2 (Nav2) case study; Nav2 is chosen because it combines topics, services, and actions in a realistic robotic application. The case studies are complemented by unit tests, Docker Compose deployment examples, offline replay, and a multi-robot scenario (Chapter 6).

1.5 | Contributions

The contributions of this thesis are:

1. An overview of existing ROS 2 monitoring approaches, the observation mechanisms they use, and the gaps that remain (RQ1).
2. A feature model that captures the commonalities and variabilities of ROS 2 monitoring solutions (RQ2).

3. A reference architecture for ROS 2 monitoring that separates observation, processing, transport, checking, feedback, and evidence handling. The architecture maps the general monitoring tasks identified in the software-monitoring literature [3] to ROS 2 systems and makes the placement of monitoring functions explicit (RQ3).
4. A monitoring model that can express several patterns found in existing work: ROS-facing collection with an external oracle as in ROSMonitoring [5], configurable monitoring services as in ROMoSu [6], and remote verification loops such as digital-twin runtime verification [7]. The same architecture also supports combinations that are not covered by these systems alone, such as passive ROS 2 topic, service, and action observation feeding either local or remote checking components (RQ3).
5. An evaluation of the infrastructure through three case studies (RQ4). A speed-limit case study demonstrates topic monitoring: velocity messages are collected, reduced to the relevant fields, and checked against a threshold in both the integrated and the split deployment. A service reset case study demonstrates service monitoring: reset requests and responses are captured through service introspection and checked for their effect on the robot state. A Navigation 2 case study demonstrates action monitoring in a realistic application: navigation goals are followed through action feedback and status, together with the topics that Nav2 uses during navigation.

The main contribution is the reusable monitoring architecture and the infrastructure that realizes it. The thesis provides the architectural layer between ROS 2 communication and property-specific checking. This layer can host deployment styles used in existing monitoring frameworks and extend them with uniform ROS 2 observations, split robot/verifier deployment, replay, and evidence handling.

1.6 | Thesis Outline

Chapter 2 explains the main ROS 2 and monitoring concepts. Chapter 3 reviews related monitoring and verification approaches. Chapter 4 presents the feature model and the reference architecture derived from it. Chapter 5 describes the prototype. Chapter 6 presents the evaluation through the case studies. Chapter 7 discusses the results and limitations. Chapter 8 concludes the thesis and describes future work.

Chapter 2

Background

This chapter explains the main concepts used in this thesis. It first introduces ROS 2 and its communication mechanisms. It then explains runtime monitoring and its basic terminology, describes how ROS 2 applications can be observed, and summarizes the main tasks of a monitoring system. Finally, it introduces feature models as a way to describe variability, and discusses deployment and transport choices.

2.1 | ROS 2 Applications

2.1.1 Nodes and the ROS Graph

ROS 2 is a middleware and software platform for building robotic applications. It provides communication and discovery approaches among components of a robotic system, and many reusable packages that help components work together to build different applications [1].

A ROS 2 application is usually divided into *nodes*. Each node has a focused task. For example, one node may publish camera data, another node may plan a route, and another node may control the motors. The active nodes and their communication links form the *ROS graph*. Nodes can run on the same computer or on different computers.

ROS 2 communication uses defined types. A type describes the fields of the data that nodes exchange, and client libraries such as `rclcpp` and `rclpy` provide the matching C++ and Python APIs. Nodes communicate through three main mechanisms: topics, services, and actions. The following subsections explain each mechanism.

2.1.2 Topics

Topics provide publish-subscribe communication. A publisher sends typed messages to a topic name. Any number of subscribers can receive these messages. The publisher does not need to know the subscribers directly. Topics are often used for sensor data, robot state, movement commands, and diagnostics [2, 8]. Figure 2.1 shows the communication pattern: one publisher sends messages to a topic, and every subscriber of that topic receives them.

Messages on a topic flow in one direction, from publishers to subscribers. Several nodes may publish to the same topic, and a node can be a publisher on some topics and a subscriber on others at the same time.

2.1.3 Services

Services provide request-response communication. A client sends a request to a named service, and the server that offers this service returns a response. Services are normally used for short operations that need a clear result, such as resetting a pose or switching a component on or off [2]. Figure 2.2 shows the pattern.

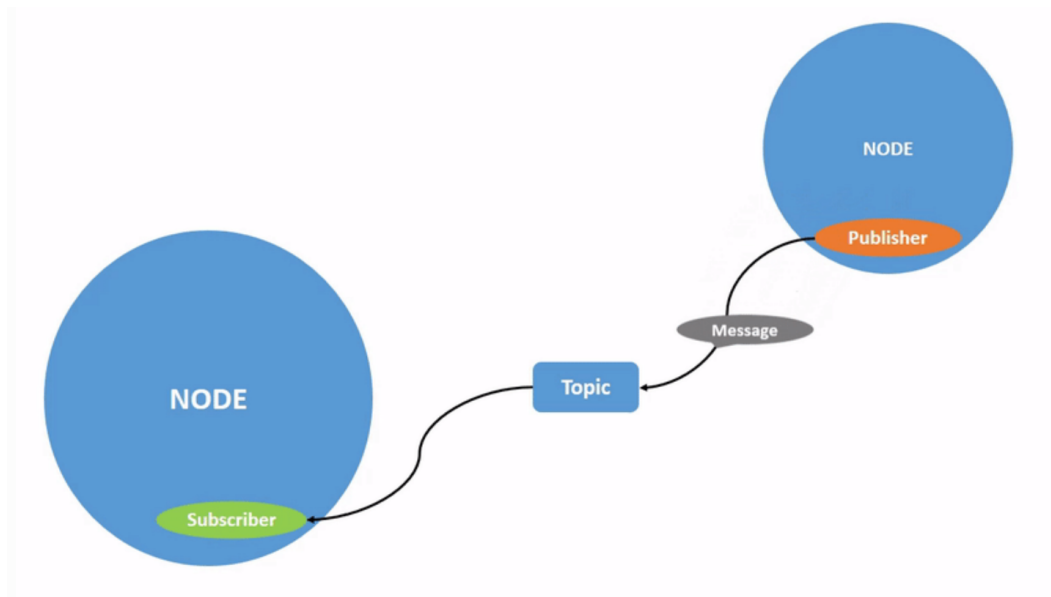


Figure 2.1: Topic communication. A publisher sends typed messages to a named topic, and all subscribers of that topic receive the messages.

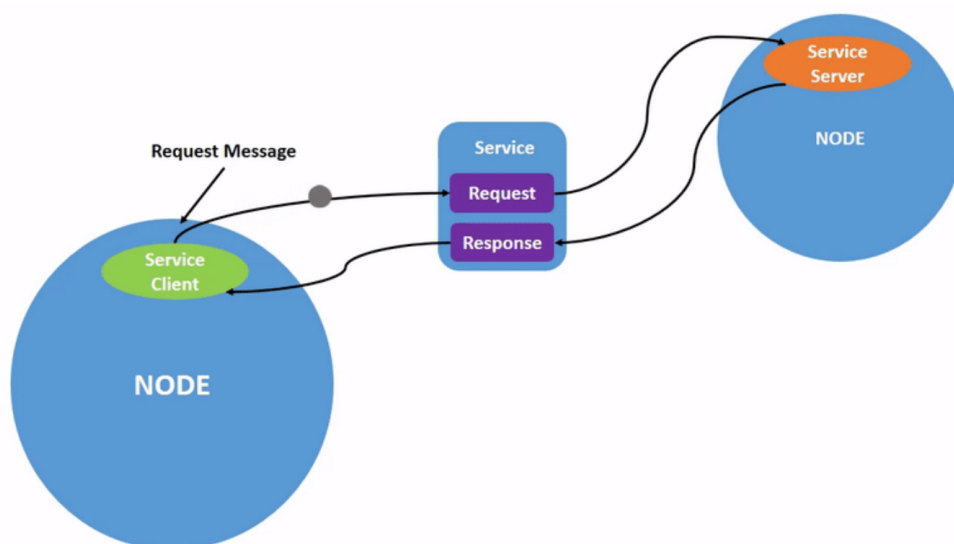


Figure 2.2: Service communication. A client sends a request to a named service, and the server returns a response to that client.

In contrast to topics, service communication is a one-to-one exchange. The request is delivered only to the server, and the response is delivered only to the client that sent the request. This property matters later in this chapter, because it limits how services can be observed.

2.1.4 Actions

Actions are used for tasks that take time to complete. An action client sends a goal to an action server. While the goal is running, the server sends feedback and status updates. The client can cancel the goal, and the server later returns a result. Navigation and robot manipulation are common examples [2, 9]. Figure 2.3 shows the pattern.

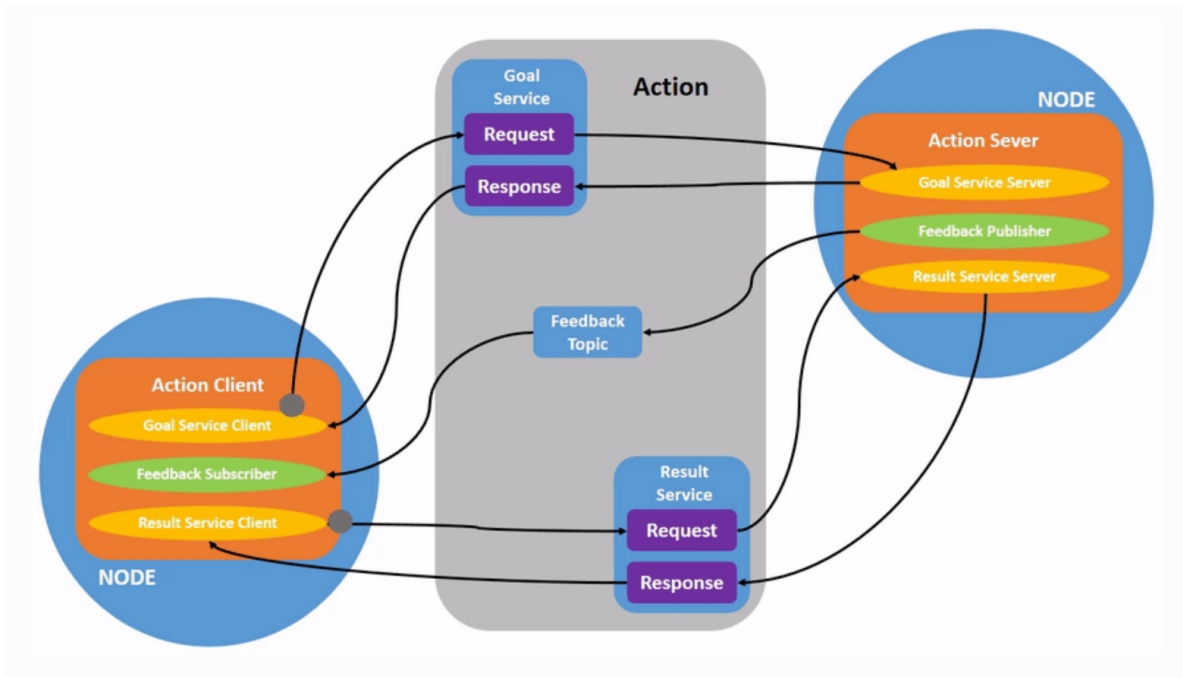


Figure 2.3: Action communication. Goal, cancellation, and result exchanges use services, while feedback and status updates use topics (dashed arrows).

Internally, an action is built from hidden topics and services. Goal, result, and cancellation communication use services. Feedback and status updates use topics [9]. An action therefore combines the one-to-one request-response pattern with continuous one-way data streams.

2.1.5 Discovery, Middleware, and Quality of Service

ROS 2 commonly uses Data Distribution Service (DDS) middleware for communication. DDS provides distributed discovery, so ROS 2 does not need the central master used in ROS 1 [1]. Nodes can discover the names and types of visible topics and services at runtime. Discovery takes time, however, and the ROS graph can change while the application is running, so the set of visible names and types is not a fixed snapshot.

ROS 2 also provides Quality of Service (QoS) settings. These settings control reliability, stored history, durability, deadlines, and other communication behaviour [10]. Two endpoints can only communicate when their QoS settings are compatible.

2.2 | Runtime Monitoring and Runtime Verification

2.2.1 Main Terms

Runtime monitoring refers to observing a system while it is running. It can support several purposes, including operational diagnosis, error detection, security monitoring, and requirements checking [3].

Runtime verification is a closely related technique in which an observed execution is checked against a specification [4]. It can report whether the observed behaviour satisfies or violates a property, but it does not establish correctness for all possible executions. In this thesis, runtime monitoring is used as the broader term for observing and processing runtime data, while runtime verification denotes the property-checking task that such monitoring can support.

The same basic roles appear in model-driven runtime-verification approaches such as ComMA, where interface and component models are used to generate monitors that consume execution traces and report violations of the specified model [11]. The thesis uses these roles in a broader monitoring infrastructure rather than in a model-generation workflow.

This thesis uses the following terms:

System under monitoring The ROS 2 application being observed.

Observation or event One captured runtime event, such as a topic message or service request.

Trace A sequence of observations used for later checking.

Property A rule or expected condition.

Monitor A component that observes the system or checks properties.

Verdict The result of a property check.

Figure 2.4 shows how these terms relate. The monitor captures observations from the system under monitoring. Based on a property, it evaluates the observed behaviour and reports a verdict to a user or to another component.

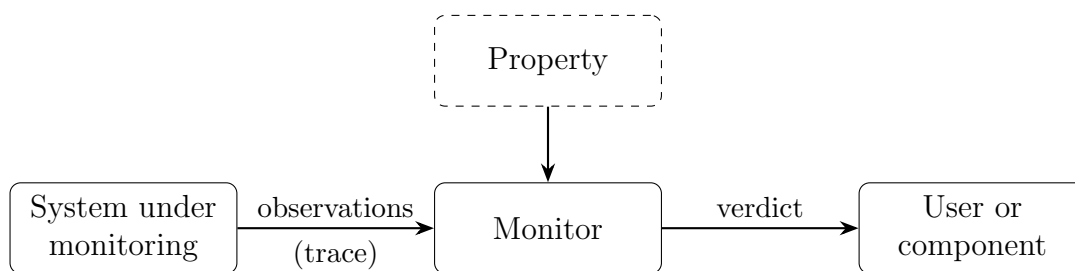


Figure 2.4: Basic runtime monitoring terminology. A monitor captures observations from the system under monitoring, checks them against a property, and reports a verdict.

2.2.2 Online and Offline Monitoring

Online monitoring processes data while the system is running. It can report problems quickly and can support immediate feedback. The design question is where this monitoring logic should run and which observations need to be processed online.

Offline monitoring checks recorded data after the run has finished. It cannot react immediately, but the same data can be replayed many times and checked with checkers that are easier to run outside the robot application.

2.2.3 Passive and Active Monitoring

A passive monitor observes and reports. It does not change the behaviour of the application. An active monitor can respond to a problem, for example by blocking a message or starting a recovery action.

2.2.4 Centralized and Distributed Monitoring

In a centralized setup, observations are sent to one main checking component. Keeping the checking state in one place simplifies the design, although the central component can become a bottleneck or a single point of failure.

In a distributed setup, observation or checking work runs in several places. This can place monitoring logic closer to the data source or closer to a verifier, but it introduces questions about message ordering, clocks, network boundaries, and failures [12]. Section 2.6.1 describes the main ways in which such monitors can be organised, and concrete deployment diagrams for the setups used in this thesis are shown in Chapter 4.

2.3 | Observing ROS 2 Applications

A monitor for a ROS 2 application first has to obtain observations. The three communication mechanisms from the previous section can each be observed, but not in the same way. This section explains the observation mechanism for each of them, and the role that types and discovery play for a monitor.

2.3.1 Observing Topics

A monitor can observe a topic by creating an extra subscription, as any other node would. This is a simple and passive method because the original communication path does not need to change. Common ROS 2 tools such as `ros2 topic echo` and `ros2 bag record` work in this way.

Topic observation still requires several choices. Some topics publish large messages or publish at a high rate, while a monitor may only need a few fields or a subset of the messages. The monitoring subscription must also use a QoS setting that is compatible with the publisher; otherwise it may receive no messages or lose data under high load. Finally, messages from different publishers may arrive in a different order from the order in which they were sent.

2.3.2 Observing Services

Services cannot be observed in the same way as topics. As explained in Section 2.1.3, a service call is a one-to-one exchange: the request travels from one client to the server, and the response travels back to that client only. A third node that wants to watch the service cannot “listen in” on this exchange. If it creates its own client for the same service, it can only make its own calls; it never sees the requests and responses of other clients.

ROS 2 solves this with *service introspection*. When introspection is enabled on the client or the server, that node publishes an event message on an additional *service event topic* for every step of a call. Each event carries metadata (such as the event type, a client identifier, a sequence number, and a timestamp) and can also carry the request or response payload [13]. Because the events are published on a normal topic, a monitor can observe them with a plain subscription, as shown in Figure 2.5.

This mechanism keeps the observation passive, but it has two consequences for a monitor. First, introspection must be enabled by the application; without it, the event topic carries no data. Second, a request and its response arrive as separate event messages, so the monitor

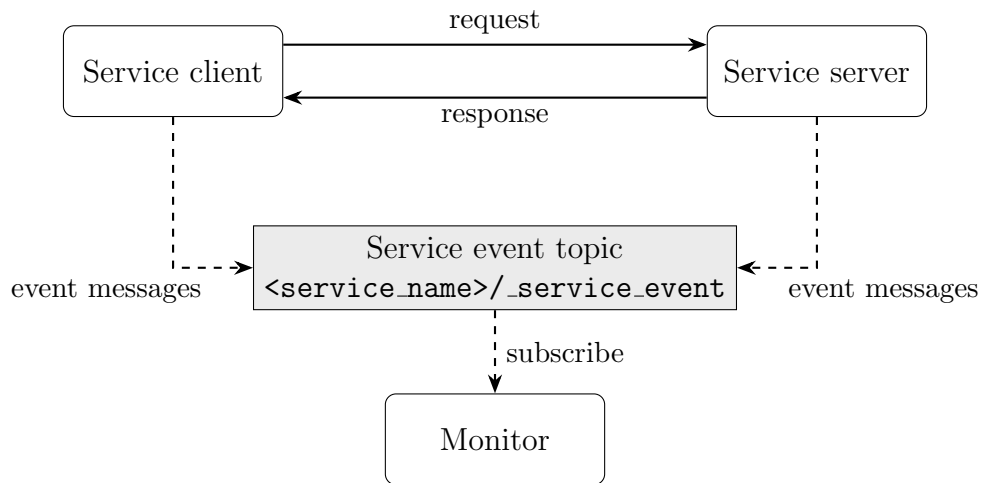


Figure 2.5: Observing a service with service introspection. The request and response are exchanged only between the client and the server (solid arrows). With introspection enabled, the client and server additionally publish event messages on a service event topic, which a monitor can observe through a normal subscription (dashed arrows).

must use the event metadata, such as the client identifier and sequence number, to connect a response to the correct request.

2.3.3 Observing Actions

Actions combine the previous two cases, because they are built from hidden topics and services. The feedback and status streams are topics, so a monitor can subscribe to them directly. The goal, result, and cancellation exchanges are services, so observing them requires service introspection on the action server or client.

2.3.4 Types and Discovery for Monitors

Type information is important for every observation mechanism above. A monitor needs the correct type before it can read and understand captured data. Discovery can help a monitor find the type of a topic or service automatically, but as explained in Section 2.1.5, discovery takes time and the graph can change at runtime, so a monitor should not assume that every source is already visible at startup.

The ROS graph also shapes what monitoring looks like in ROS 2. A monitoring node can often join the graph without changing the original application, which makes passive observation natural. At the same time, the behaviour of the full robot is spread over many nodes, so a monitor must select useful interfaces and keep enough context to connect related events.

2.4 | Main Monitoring Tasks

A monitoring system usually performs several different tasks [3]. Keeping these tasks separate makes the infrastructure easier to configure and extend.

2.4.1 Collecting Data

The first task is to obtain data from the running system. In ROS 2, this can mean subscribing to topics [5, 14], reading service events [14], observing action feedback [1], sniffing network traffic [15], using lower-level tracing tools [16], or even instrumenting the application code [17, 18] (Section 2.3).

Each method has different limits. Normal subscriptions are easy to add, but they only show what the monitor receives. Service introspection needs support from the client or server. Network sniffing observes packets or middleware traffic outside the application process, so it can see communication without adding a ROS 2 subscription or modifying the nodes. However, the captured traffic often has to be decoded and mapped back to ROS 2 names, types, and application meaning. Lower-level tracing observes execution events inside the ROS 2 or operating-system stack, such as callbacks, scheduling, or middleware tracepoints. This can show detailed timing and causal information, but it may still not include the application meaning of a message. Instrumenting the code gives the most direct access, but it changes the application itself.

2.4.2 Processing Data

Raw data is not always ready for checking. A monitor may need to select fields, rename values, add timestamps, or connect events to a session. It may also need to reduce the amount of data.

2.4.3 Routing and Storage

After processing, raw data is wrapped as event trace, and needs to be sent to the correct destination. For example, one event trace may be written to a file, sent over a network, and passed to a checking component. Adding or changing such destinations should not require changes to the components that collect the data.

Storage also supports later analysis. A stable data format makes it easier to replay a run or to compare results from different checking tools.

2.4.4 Checking and Verdicts

Checking decides whether observed data follows an expected property. Different tools need different input formats. A simple threshold checker may need one number, while a temporal-logic checker may need a sequence of timestamped values.

2.4.5 Using the Results

A verdict is only useful when it reaches the right user or component. It may be printed during development, stored in a file, sent to a dashboard, or passed to another service.

2.5 | Variability and Feature Models

Systems in the same domain usually share a common core but differ in many details. Building one configurable infrastructure for such a domain only works when these common and variable parts are identified first: the common parts can be implemented once, while the variable parts must become configuration choices. This requires a systematic way to describe what can vary and which combinations are allowed.

Feature-Oriented Domain Analysis (FODA) provides such a description [19]. In FODA, the capabilities of the systems in a domain are described as *features*, organized in a tree called a *feature model* or feature diagram. The tree shows which features are always present, which are optional, and which exclude each other, and it can be extended with cross-tree constraints between features in different branches [20].

Figure 2.6 shows a small example feature model of a computer product line. The notation is read as follows:

- A **mandatory** feature (filled circle) is part of every product in which its parent appears. Every computer has a processor and memory, as well as some storage and ports.

- An **optional** feature (empty circle) may be included or left out. A computer may or may not include a dedicated graphics card or a more powerful power supply.
- An **alternative** group (empty arc) allows exactly one child to be chosen. The processor is from Intel or AMD, but only one processor family is selected for a particular model.
- An **or** group (filled arc) allows one or more children to be chosen. A computer can include SSD storage, hard disk storage, or both.
- A **cross-tree constraint** (dashed arrow) restricts combinations across branches. A dedicated graphics card requires stronger power supply. Constraints can also exclude combinations, for example when two features cannot be used together.

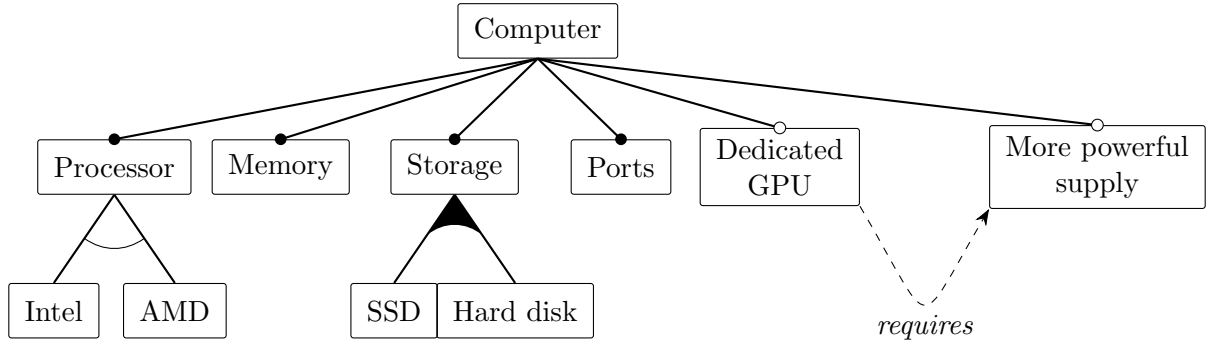


Figure 2.6: Example feature model of a computer product line. Filled circles mark mandatory features, empty circles mark optional features, an empty arc marks an alternative (exactly one) group, a filled arc marks an or (one or more) group, and the dashed arrow is a cross-tree constraint.

A concrete product is a selection of features that respects all these rules. In this thesis, the same notation is used in Chapter 4 to describe the commonalities and variabilities of ROS 2 monitoring solutions.

2.6 | Deployment and Transport

The monitoring tasks described earlier in this chapter do not have to run in one place. Observation, checking, and reporting can be placed at the same location or spread over several machines. Francalanza et al. compare the main ways in which monitors for decentralised and distributed systems can be organised [12]. This section summarizes these monitoring organisations and then discusses the transport mechanisms that can connect monitoring components at different locations.

2.6.1 Monitoring Organisations

Figures 2.7 to 2.10 show the four monitoring organisations described by Francalanza et al. [12]. The figures use their notation: circles are system processes (P_i), hexagons are traces (T_i), squares are monitors (M_i), and each surrounding rectangle is a location (l, k, g, h), such as a machine or a robot. Dotted arrows show processes contributing events to a trace, dash-dotted arrows show monitors reading a trace, and solid arrows show interaction between components.

In a *traditional* (centralised) monitoring setup (Figure 2.7), all processes run at a single location. They produce one local trace, and a single monitor at the same location analyses it. Because everything runs under one clock, the trace is a total order of the events that

actually happened. This is the simplest organisation, but the whole monitoring task rests on one monolithic monitor at one location [12].

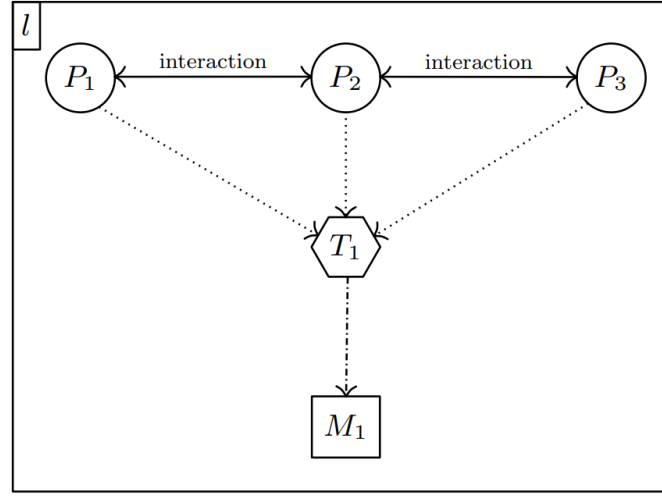


Figure 2.7: Traditional (centralised) monitoring: the processes at one location produce a single trace that one monitor analyses (adapted from [12]).

In a *decentralised* monitoring setup (Figure 2.8), the events are still governed by a single global clock and form one synchronised, totally ordered trace, but the monitoring work is decomposed into several monitors. The monitors can interact to combine their partial results, and processes and monitors may be placed at different locations that share the global clock. This decomposition reflects that a global property can often be split into smaller properties that cooperating monitors check independently [12].

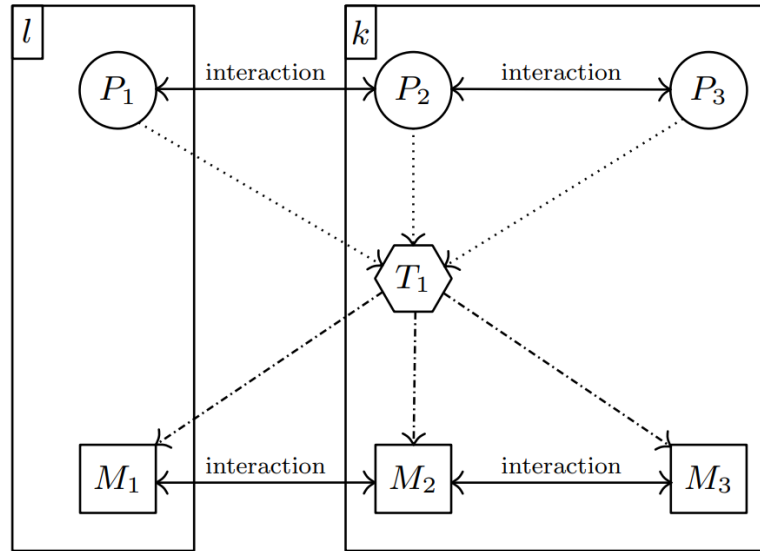


Figure 2.8: Decentralised monitoring: several monitors, possibly at different locations that share a global clock, process a synchronised trace and can interact with each other (adapted from [12]).

The remaining two organisations target truly distributed settings, where processes run at several locations without a global clock. Each location then produces its own local trace, and the events of different traces can only be partially ordered. In an *orchestrated* monitoring setup

(Figure 2.9), the analysis is still performed centrally: the monitors run at a remote location and access the trace events produced at the other locations. Centralising the analysis keeps the monitor logic simple, but the trace data has to be transmitted remotely. This increases the communication overhead as the system grows, and it can expose private trace data on the network and make the central monitor a single point of attack [12].

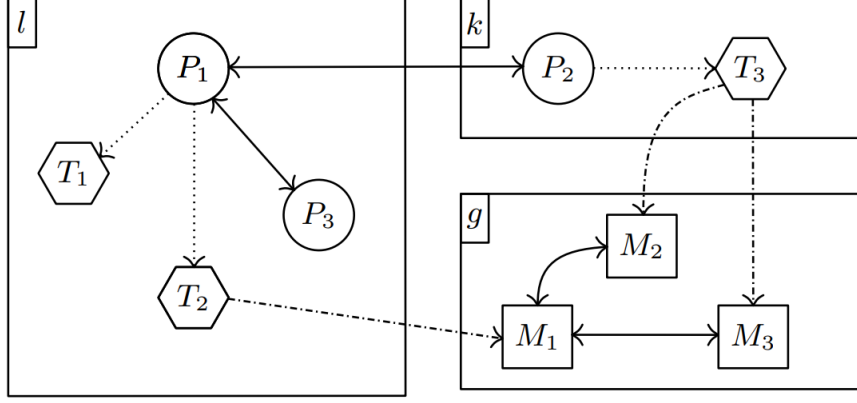


Figure 2.9: Orchestrated monitoring: traces are produced locally at the observed locations, while remote monitors at a central location perform the analysis (adapted from [12]).

In a *choreographed* monitoring setup (Figure 2.10), the verification work is instead pushed to the locations where the traces are produced. Local monitors analyse local traces and, when they contribute to the same global property, coordinate with each other. This reduces data exposure and communication overhead, but it is more complex to set up: the properties have to be decomposed into coordinated local monitors, and the monitored locations must allow the extra local computation [12].

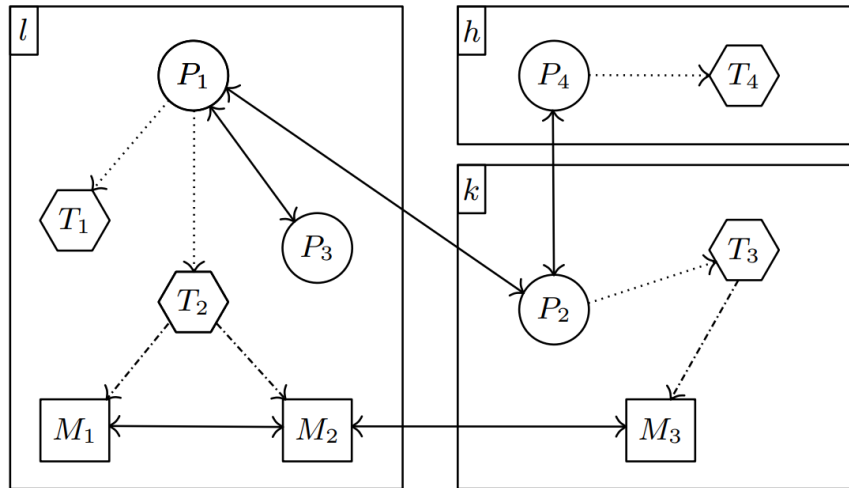


Figure 2.10: Choreographed monitoring: monitors run at the locations where the traces are produced and coordinate with each other when needed (adapted from [12]).

These organisations frame the deployment choices for a ROS 2 monitoring infrastructure. A single robot with an on-board monitor is a traditional setup. As soon as observations are checked on another machine, or several robots are observed together, the infrastructure moves towards an orchestrated or choreographed organisation, and the partial ordering, communication, and

data exposure trade-offs above apply. Section 4.7 relates the deployment forms supported by this thesis to these organisations.

2.6.2 Transport Protocols

When monitoring components run in different processes or on different machines, as in the orchestrated and choreographed organisations above, they need a transport mechanism to exchange data. Many protocols can fill this role. The components could communicate through the DDS middleware that ROS 2 itself uses, through plain TCP or UDP sockets, through web protocols such as HTTP or WebSocket, or through message brokers based on protocols such as AMQP, Kafka, or MQTT. These options differ in how they handle addressing, delivery guarantees, and decoupling between sender and receiver.

MQTT is a publish-subscribe messaging protocol that is commonly used in robotics and Internet-of-Things systems, because it is simple, lightweight, and has wide tool support. Publishers send messages to named topics through a broker, and subscribers receive messages from the topics they select. MQTT 5.0 is an OASIS standard [21]. Because the broker decouples publishers from subscribers, neither side needs to know the network address of the other. This thesis uses MQTT as the transport in the implementation of the case studies; the transport itself remains a configurable choice in the infrastructure.

In a monitoring setup that uses MQTT, a robot-side monitor publishes serialized observations to a configured MQTT topic or topic prefix. A remote verifier subscribes to that topic filter and receives the data without needing a ROS 2 dependency or a direct connection to the robot process. Verdicts can also be published to a separate MQTT topic, which keeps runtime data and checking results distinct.

MQTT also gives a configurable delivery trade-off through Quality of Service (QoS) levels. QoS 0 sends a message at most once and is useful when occasional loss is acceptable. QoS 1 retries until the broker acknowledges the message, which can duplicate messages and therefore requires identifiers to make duplicates recognizable. QoS 2 adds a stronger exactly-once exchange at the expense of a more involved protocol exchange. The infrastructure keeps the session and record identifier inside each observation so that the transport does not have to carry the semantic identity of an observation.

Chapter 3

Related Work

There are many ways to monitor a robotic system. Some tools observe ROS messages, some generate checking code from requirements, and others trace the operating system or network. These approaches solve different parts of the monitoring problem. This chapter reviews the work that is most relevant to this thesis and explains how the infrastructure developed here differs.

3.1 | How the Approaches Are Compared

The word “monitoring” has a broad meaning. For example, a tracing tool may explain why a callback was slow, while a runtime-verification tool may report that a safety rule was broken. Both tools observe the running system, but their goals are different.

The comparison follows the questions that kept recurring during the domain analysis: which part of the system is observed, which ROS interfaces are covered, how properties are checked, what can be configured or extended, where collection and checking run, and what the approach is mainly trying to achieve.

These questions follow the general monitoring tasks identified by Rabiser et al., such as collection, processing, checking, routing, and using the results [3].

3.2 | ROSMonitoring

ROSMonitoring is a runtime-verification framework for ROS [5]. It adds monitor nodes between selected publishers and subscribers. A YAML file describes the topics that should be monitored. The monitor changes ROS messages into events and sends them to an external *oracle*. The oracle checks the events against a property.

The useful idea for this thesis is the boundary between ROS message handling and the external oracle. ROSMonitoring shows that the checker does not have to live inside the ROS-facing code. In this thesis, that boundary is combined with passive subscriptions and service introspection, so records can be routed through separate processing, transport, checking, and output stages without placing a proxy in the normal application communication path.

ROSMonitoring 2.0 adds support for services and studies message ordering [14]. Some service support is available for ROS 2. Full ROS 2 support and action monitoring are left as future work. Ordering is an important topic because messages may not arrive at the monitor in the same order in which they were published.

For this thesis, ROSMonitoring 2.0 is mainly a reminder that services and ordering become visible as soon as monitoring moves beyond plain topic streams. The prototype keeps ordering support modest and concentrates on a uniform record and pipeline for heterogeneous ROS 2 observations.

3.3 | ROMoSu

Stadler et al. describe several practical problems when adding monitoring to a ROS application. Developers need to understand the system, select useful data, choose collection rates, add checks, and present the results [22].

ROMoSu provides a flexible framework for these tasks [6]. Users can create reusable monitoring configurations, select topics and message fields, choose collection rates, and connect external services. ROMoSu also includes a user interface, a dashboard, and configuration storage.

ROMoSu is the closest work in spirit because it treats monitoring as something that should be configured, not hand-written from scratch each time. Its scope is broader on the user-facing side: it includes a dashboard, storage, and a larger platform around the monitoring process. The prototype here concentrates on the internal infrastructure stages - collectors, transformers, converters, verdict services, sources, and exporters - and gives direct attention to ROS 2 services, actions, and split robot/verifier deployment.

3.4 | Generated Monitors for ROS 2

Perez et al. present a process for generating ROS 2 runtime monitors from requirements [17]. The user writes structured requirements in FRET. These requirements are translated to temporal logic and then to Copilot monitor code through Ogma. The final monitor is compiled to C99 and placed in a generated ROS 2 package.

This is a requirements-to-monitor pipeline. The prototype starts one step earlier in the data path: it collects ROS 2 observations and prepares them for whatever checker is selected later. A generated FRET–Ogma–Copilot monitor could fit behind the verdict-service interface. The plugin examples implemented for this thesis are simpler custom converter and verdict-service classes.

3.5 | RTAMT

RTAMT is a library for monitoring Signal Temporal Logic (STL) properties [23]. It supports online and offline checking and can work with ROS 2. RTAMT can return a robustness value, which shows how strongly a signal satisfies or violates a property.

RTAMT helped clarify the role of converters in our design. STL monitoring needs timestamped signals, while another checker may want a different record shape. For that reason, the prototype keeps conversion separate from the verdict service. The thesis demonstrates that boundary with threshold-style and custom property plugins; an RTAMT-specific plugin remains future work.

3.6 | Tracing and Performance Monitoring

`ros2.tracing` provides low-overhead tracing for ROS 2 [16]. It uses LTTng and tracepoints in the ROS 2 core. Developers can study callback execution, message flow, latency, and operating system events.

`ros2.tracing` sits below the level used in this thesis. It is the right tool when the question is about callback execution, message flow, or middleware timing. The prototype stays with typed application data. In practice, the two levels are complementary: a property verdict can point to a suspicious run, and a trace can help explain the execution details behind it.

3.7 | Network-Level Monitoring and Security

ROS-FM observes ROS network traffic using eBPF and XDP [15]. It can collect network information, apply security rules, and support distributed visualization. Because it works below the ROS application layer, it can observe communication without adding a normal ROS subscriber for every topic.

This kind of monitor sees traffic without adding a ROS subscriber. The difficult part is reconstructing application meaning from packets. The design in this thesis stays at the application level, where ROS 2 messages already carry the type information that converters and checkers need.

ROSIDS23 and ROSPaCe provide datasets for ROS intrusion-detection research [24, 25]. ROSPaCe combines operating-system, network, and ROS 2 service data. These datasets show that security monitoring may need information from several system layers. Their main goal is security detection. In this chapter they mainly mark the boundary between security datasets and the more general monitoring infrastructure developed here.

3.8 | Containerized and Distributed Verification

Aldegheri et al. present a containerized verification environment for ROS [26]. Generated monitors run as ROS nodes in Docker containers. The work also studies where monitors should run between edge and cloud systems.

This work connects to the split deployment in this thesis because it treats placement as part of the monitoring design. The prototype here leaves placement as an explicit configuration choice: it provides integrated and split examples and keeps the transport interface replaceable.

Distributed runtime verification is a wider research field. It studies how monitoring can work when the monitored system and monitors are spread across different machines [12]. Important problems include clock differences, message delays, failures, and the division of a property between local monitors.

The split mode in this thesis is deliberately simpler than distributed runtime verification. Collection and checking can run on different machines, but checking is still logically centralized at the verifier.

3.9 | Digital-Twin Runtime Verification

Betzer et al. use a digital twin for runtime verification of an autonomous mobile robot under uncertainty [7]. Their monitors are synthesized from TeSSLa specifications and run with a cloud-located digital twin. The digital twin estimates the robot state, checks safety and performance properties, and can intervene when a property is about to be violated.

The interesting contrast is that the digital twin gives the checker a richer world model. The prototype in this thesis stays closer to the ROS 2 communication layer and builds the path from typed ROS 2 observations to checking components.

3.10 | Field-Based Testing Guidance

Caldas et al. provide guidance for runtime verification and field-based testing of ROS systems [18]. They explain that real-world conditions cannot always be fully reproduced in simulation. Robotic software should therefore support data collection, isolation, monitoring reactions, and the use of field evidence.

This paper helped frame the practical side of the record format. If field data is later used as evidence, the record needs enough context to connect an observation to a source, a session,

and a verdict. The prototype supports that kind of evidence trail through JSONL and MQTT outputs. The wider field-testing process, automatic recovery, and test generation remain outside its scope.

3.11 | Summary Comparison

Table 3.1 gives a short comparison of the main approaches.

Table 3.1: Comparison of selected related work.

Approach	Main focus	Checking and deployment	What we take from it
ROSMonitoring / 2.0	Intercepting ROS messages and services	External oracle; online and offline checking	Useful oracle boundary; this prototype keeps observation passive and includes action progress.
ROMoSu	Flexible and reusable ROS monitoring configurations	External services, user interface, and dashboard	Strong example of configurable monitoring; this work narrows in on the plugin-based pipeline underneath such a platform.
FRET-Ogma-Copilot	Generating monitors from structured requirements	Generated real-time checking code	Good candidate for a future verdict-service backend; not part of the current prototype.
RTAMT	Checking timestamped signals against STL properties	Online and offline STL monitoring	Motivates the converter boundary because STL checkers expect signal-shaped input.
<code>ros2_tracing</code>	Low-level ROS 2 and operating-system tracing	Trace analysis instead of property verdicts	Occupies a lower layer than the typed application records used here.
Digital-twin RV	Digital twin for robot monitoring under uncertainty	TeSSLa monitors with a digital-twin feedback loop	Shows how remote checking can use a richer state model; this thesis keeps the state source in ROS 2 records.
ROS-FM	Efficient network and security monitoring	Network metrics, security rules, and visualization	Useful contrast with application-level monitoring because packet data must be given ROS meaning again.

Table 3.1: Comparison of selected related work (continued).

Approach	Main focus	Checking and deployment	What we take from it
Containerized verification	Portable generated monitors and monitor placement	Docker and edge-to-cloud deployment	Informs the deployment view; the prototype leaves placement and transport as explicit configuration choices.
This thesis	Configurable collection of typed ROS 2 observations	Pluggable checking in integrated or split deployments	Connects topics, service events, and action progress to reusable processing, transport, checking, and output stages.

3.12 | Capability Matrix

The qualitative comparison above is complemented by two capability matrices. The symbols have the following meaning: ✓ means direct support, ● means limited support or support through a specific condition, and ✗ means that the capability is not a main feature of the approach. In the observable-element matrix, “Life.” refers to node lifecycle or node-state information, and “Kernel” refers to low-level operating-system or tracing events.

Table 3.2: Observable-element support in related work.

Approach	ROS version	Topics	Srv.	Act.	Par.	Life.	Kernel
ROSMonitoring	ROS 1	✓	✗	✗	✗	✗	✗
ROSMonitoring 2.0	ROS 1/2	✓	✓	✗	✗	✗	✗
ROMoSu	ROS 1	✓	✗	✗	✗	✗	✗
Generated ROS 2 monitors	ROS 2	✓	✗	✗	✗	✗	✗
ros2_tracing	ROS 2	✓	●	✗	✗	✓	✓
RTAMT	ROS 1/2	✓	✗	✗	✗	✗	✗
Digital-twin RV	ROS 1	✓	✗	✗	✗	✗	✗
This thesis	ROS 2	✓	✓	●	✗	✗	✗

Table 3.3: Checking formalism and execution mode in related work.

Approach	Formalism or checking mechanism	Online	Offline
ROSMonitoring	External oracle, for example Reelay	✓	✗

Table 3.3: Checking formalism and execution mode in related work (continued).

Approach	Formalism or checking mechanism	Online	Offline
ROSMonitoring 2.0	Runtime Monitoring Language and past-time MTL	✓	✓
ROMoSu	External services and Esper CEP	✓	✗
Generated ROS 2 monitors	FRETISH requirements and generated Copilot monitors	✓	✗
ros2.tracing	Trace collection and analysis, not a property language	✓	✓
RTAMT	Signal Temporal Logic and robustness monitoring	✓	✓
Digital-twin RV	TeSSLa specifications with a digital twin	✓	✗
This thesis	Configurable converter and verdict-service plugins	✓	✓

3.13 | Research Gap

After reviewing these approaches, the missing piece is not another property language. The gap is the infrastructure between ROS 2 communication and property-specific checking: a way to collect several kinds of ROS 2 observations, keep their context, process them uniformly, and then hand them to different checkers.

The combination pursued here is passive collection from topics, service events, and action feedback/status; one common record format with source, time, session, and identity information; configurable data reduction before storage or transport; and separate routing, checking, and output stages. Plugins then make the property-specific parts replaceable, while the same checking chain can be used in integrated and split deployments.

The contribution of this thesis is this combination as one reusable infrastructure. Its novelty lies in the uniform record format and pipeline that connect topics, service events, and action progress to the same configurable checking chain in both integrated and split deployments.

Chapter 4

Design

This chapter explains how we designed the monitoring infrastructure. The design starts from the feature model that we developed to answer RQ2. We used the model as a map of the design space rather than as an implementation checklist: it made clear which parts of ROS 2 monitoring could reasonably fit into one thesis prototype and which parts should remain outside the scope.

4.1 | Feature Model as Design Input

Figure 4.1 shows the feature model. It organizes ROS 2 monitoring along several dimensions: observable elements, monitoring architecture, monitoring mode, system configuration, and implementation paradigm. When we built this model, the main lesson was that “monitoring ROS 2” is too broad to be treated as one feature. A monitor may observe topics, services, actions, parameters, node states, or middleware events. It may run centrally or in several places. It may be passive or active. It may be implemented as a normal ROS 2 monitor node, as an inserted decorator, as a network tap, or through lower-level middleware and tracing tools.

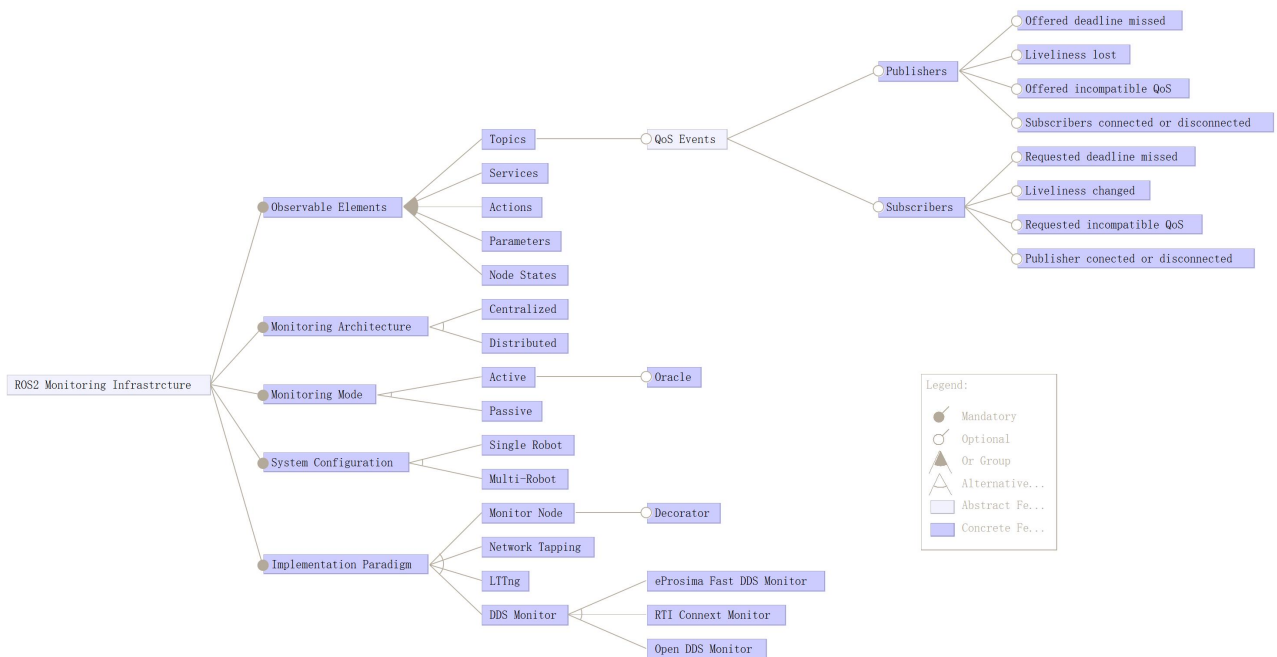


Figure 4.1: Feature model of the ROS 2 monitoring infrastructure domain, using the notation introduced in Section 2.5.

The concrete prototype implements one path through this map: application-level data that can be collected passively, represented in a common format, and configured without rewriting the monitoring core.

4.2 | Selected Feature Coverage

Table 4.1 summarizes how the implemented prototype relates to the feature model. The table distinguishes between features that are implemented directly, features that are represented by the demonstration scenarios, and features that remain outside the current design.

Feature dimension	Selected coverage	Not covered in the prototype
Observable elements	Topics, service request and response events, and action feedback and status are collected as typed ROS 2 observations.	Parameters, node lifecycle states, DDS-level QoS events, network packets, and a complete action trace with goal, cancellation, and result events.
Monitoring architecture	The infrastructure supports an integrated setup and a split robot/verifier setup. Demo scenarios also exercise hybrid, multi-robot, global, choreographed, and offline replay organizations.	Automatic monitor placement, fault-tolerant distributed verification, global message ordering, and clock synchronization guarantees.
Monitoring mode	Monitoring is passive: the prototype observes data and produces records and verdicts.	Active intervention, automatic recovery, message blocking, or changes to robot behaviour.
System configuration	Single-robot and multi-robot configurations can be expressed with the same YAML configuration concepts and record format.	Automatic generation of the full monitoring configuration from a system model or graphical tool.
Implementation paradigm	The prototype is implemented as ROS 2-facing monitor nodes plus optional verifier-side processes.	Decorator-based interception, network tapping, LTTng tracing, and DDS-specific monitor implementations.

Table 4.1: Selected prototype coverage with respect to the feature model.

This coverage is intentionally narrower than the full feature model. The goal is not to cover every possible monitoring technique. The goal is to build and validate a reusable path from ROS 2 application observations to checking components, while keeping the parts that usually change between applications configurable.

4.3 | Design Requirements

The selected feature coverage led to a few design requirements that shaped the prototype. The most important one is the common record format. Topics, service events, and action feedback/status should not force later components to deal with three unrelated data shapes. The record therefore has to carry not only the payload, but also the source, timestamp, session, and identity information needed to connect observations with later verdicts.

We also wanted processing to happen early in the pipeline. Raw ROS 2 messages can be large or frequent, and in many cases the checker only needs a few fields or only needs records when values change. This pushed field extraction, throttling, and filtering into the infrastructure before storage or transport.

The remaining requirements are about keeping changeable parts apart. Collection needs ROS 2 type and communication knowledge without being tied to one checking language. A converter may adapt records for a checker, while a verdict service contains the property-specific logic. YAML configuration covers common choices, and plugins cover project-specific conversion, checking, input, and output behaviour. The same chain should then be usable both when everything runs in one ROS 2-facing process and when records cross a transport boundary to a separate verifier.

4.4 | Infrastructure Overview

The infrastructure is organized as a dataflow pipeline. A collector observes one ROS 2 source and creates a common data record. A transformer pipeline can reduce or modify the record, for example by selecting fields, limiting the rate, or emitting only changed values. Dispatchers then route the record to data exporters and to optional checking chains.

Checking is deliberately separated into two steps. A converter prepares a data record for one property-specific checker. A verdict service evaluates the prepared data and returns a structured verdict. Verdict exporters then deliver the result to standard output, a file, MQTT, or another configured destination. This separation keeps the ROS 2 collection layer independent from a particular verification language or checking algorithm.

Figure 4.2 shows the main pipeline stages. The same chain is used for topic messages, service events, and action feedback or status records, so a new source type does not require a new checking architecture.

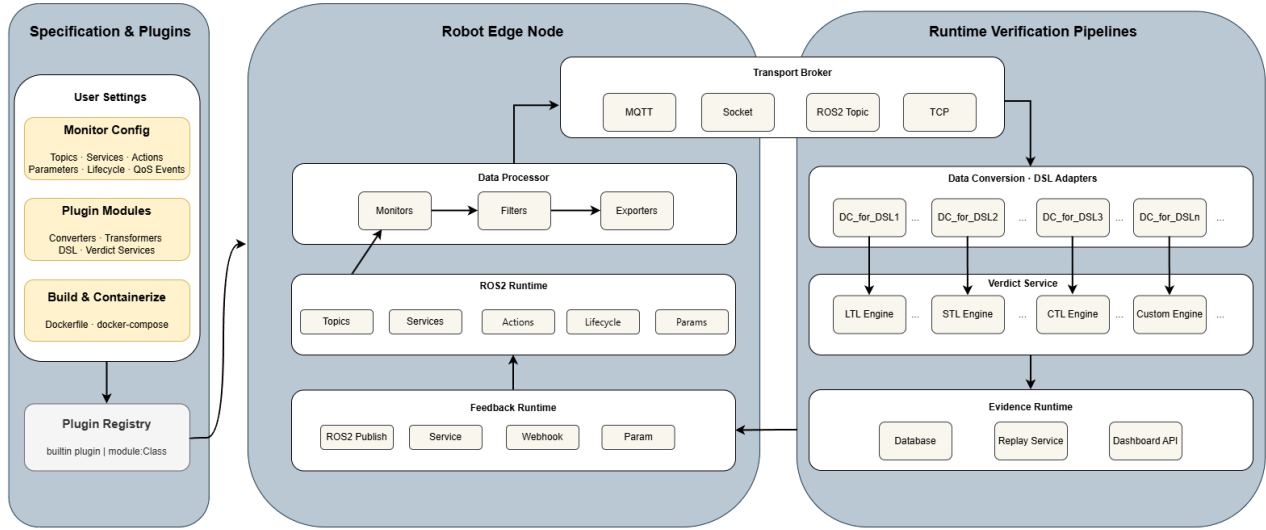


Figure 4.2: Main stages of the monitoring pipeline. Collectors create common data records, transformers process them, dispatchers route them, converters adapt them for checkers, and verdict exporters publish the checking results.

4.5 | Pipeline Roles

The implementation chapter uses several role names repeatedly. They have the following meanings in the design:

Collector A ROS 2-facing component that subscribes to one configured topic, service event stream, or action feedback/status stream and turns each observation into a data record.

Transformer pipeline An ordered set of processing steps that can remove fields, throttle records, or suppress unchanged values before data is stored, transported, or checked.

Dispatcher A routing component that fans a record out to one or more downstream exporters or checking chains.

Data exporter A sink for data records, such as a JSONL file, standard output, or MQTT topic.

Source A verifier-side input adapter that reads data records from a transport or replay file and injects them into the same checking chain.

Converter A property-specific adapter that projects a data record into the input shape expected by a checker.

Verdict service A checker that evaluates converted data and returns a structured verdict when a property state is observed.

Verdict exporter A sink for verdicts, such as standard output, a JSONL file, or an MQTT verdict topic.

This naming keeps the infrastructure precise: collectors and sources introduce records, transformers change records, dispatchers route records, converters adapt records for property logic, and verdict services perform the actual checking.

4.6 | Common Data Record

The common data record is the central abstraction of the infrastructure. Every captured topic message, service event, and action observation is converted into this format before it reaches processing, transport, or checking. Records are serialized as JSONL, with one JSON object per line. Empty optional fields are omitted from the serialized output to keep files and MQTT payloads compact.

Table 4.2 summarizes the fields used by the record. The field names follow the implementation and the detailed record specification in `docs/datarecord_spec.md`.

Field	Presence	Purpose
<code>_type</code>	All records	Identifies the record kind: session start, data, session end, or error.
<code>session_id</code>	All records	Groups all records produced during one monitoring run.
<code>record_id</code>	Data records	Gives one captured observation a stable identity for correlation with later verdicts.
<code>source_type</code>	Data records	States whether the observation came from a topic, service, or action.
<code>source_name</code>	Data records	Stores the logical ROS 2 name, such as <code>/cmd_vel</code> , <code>/set_bool</code> , or <code>/navigate_to_pose</code> .
<code>source_node</code>	When known	Records the publisher or server node name found during collection.
<code>msg_type</code>	Data records	Preserves the ROS 2 type identifier needed by downstream converters.
<code>phase</code>	Services/actions	Distinguishes request, response, feedback, status, goal, or result phases.
<code>timestamp</code>	All records	Stores the monitor-side wall-clock time when the record was created.
<code>ros_timestamp</code>	When available	Preserves the timestamp carried by the original ROS 2 message or event.
<code>data</code>	All records	Contains the monitoring configuration, captured payload, session statistics, or error details, depending on <code>_type</code> .
<code>metadata</code>	Data records	Stores infrastructure context such as sequence number, QoS profile, applied transformers, service event type, client identifier, and service sequence number.

Table 4.2: Schema of the common data record used throughout the infrastructure.

The record format is deliberately uniform across ROS 2 communication methods. Topics have no phase because each message is a complete unit. Service events are unwrapped into request and response records, with the service event metadata kept for correlation. Actions use feedback and status records directly, while goal and result phases can be represented when

service introspection for those action services is available. The same `session_id`, `record_id`, `source_type`, and `source_name` fields therefore identify observations regardless of whether they originated from a topic, service, or action.

4.7 | Deployment Topologies

The same pipeline can run in two main deployment forms. They are the concrete instances, in this infrastructure, of the general monitoring organisations introduced in Section 2.6.1.

In the integrated deployment, collection, processing, checking, and output run in one ROS 2-facing process. This form is useful for local development and for properties that can be checked close to the observed application. It corresponds to the traditional organisation in Figure 2.7: the application and the complete monitoring pipeline share one location and one clock, so the records form a totally ordered trace that a single monitor process analyses.

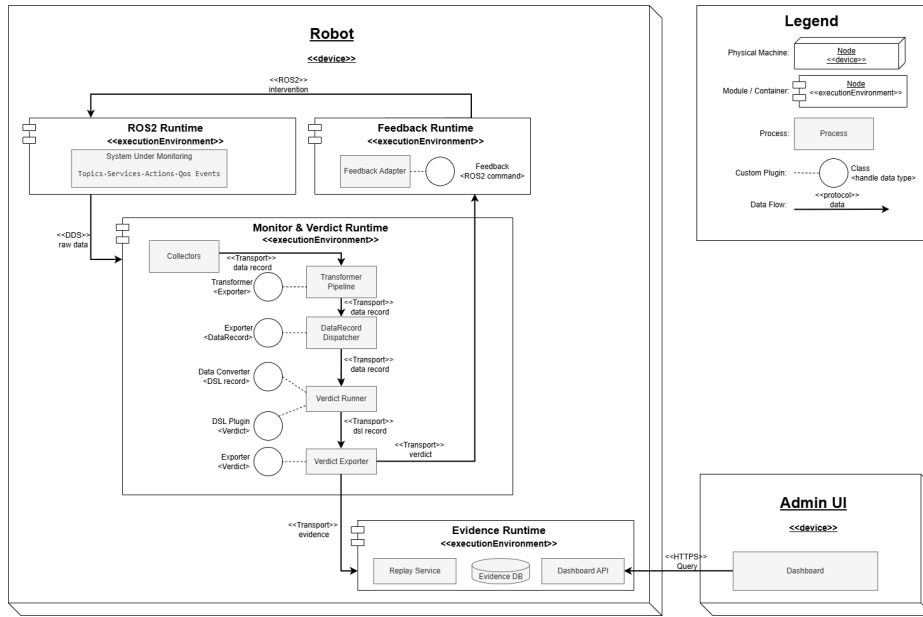


Figure 4.3: Integrated deployment topology. Collection, processing, checking, and verdict export are placed in the same ROS 2-facing monitor runtime.

In the split deployment, a robot-side monitor collects and processes observations and sends data records to a verifier-side process. The verifier uses a source adapter to read those records and then executes the same converter, verdict service, and verdict exporter chain that is used in the integrated deployment. The prototype implements file replay and MQTT for these flows, but transport remains an exporter/source choice rather than part of the checking logic.

The split deployment is closest to the orchestrated organisation in Figure 2.9: the trace is produced at the observed location, while the analysis runs on a remote verifier, so the orchestration trade-offs from Section 2.6.1, communication overhead and data exposure on the network, apply to it. Because the robot-side transformer pipeline can already filter and reduce the records before they are transmitted, the split deployment also borrows the main advantage of the choreographed organisation: part of the monitoring work stays at the location where the trace is produced, which lowers the amount of data that crosses the network.

Figure 4.5 shows the runtime order of the main pipeline interactions. The sequence is the same in both deployment forms until a data exporter or source crosses the transport boundary.

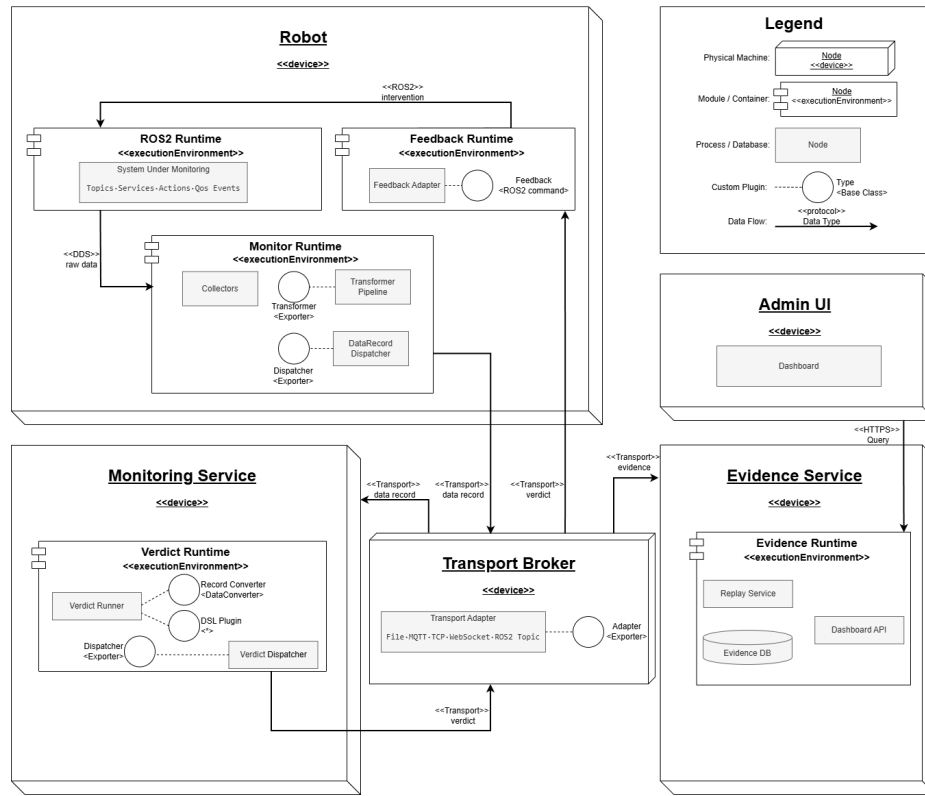


Figure 4.4: Split deployment topology. The robot-side monitor publishes data records through a transport boundary, while a verifier-side process runs the checking chain and exports verdicts.

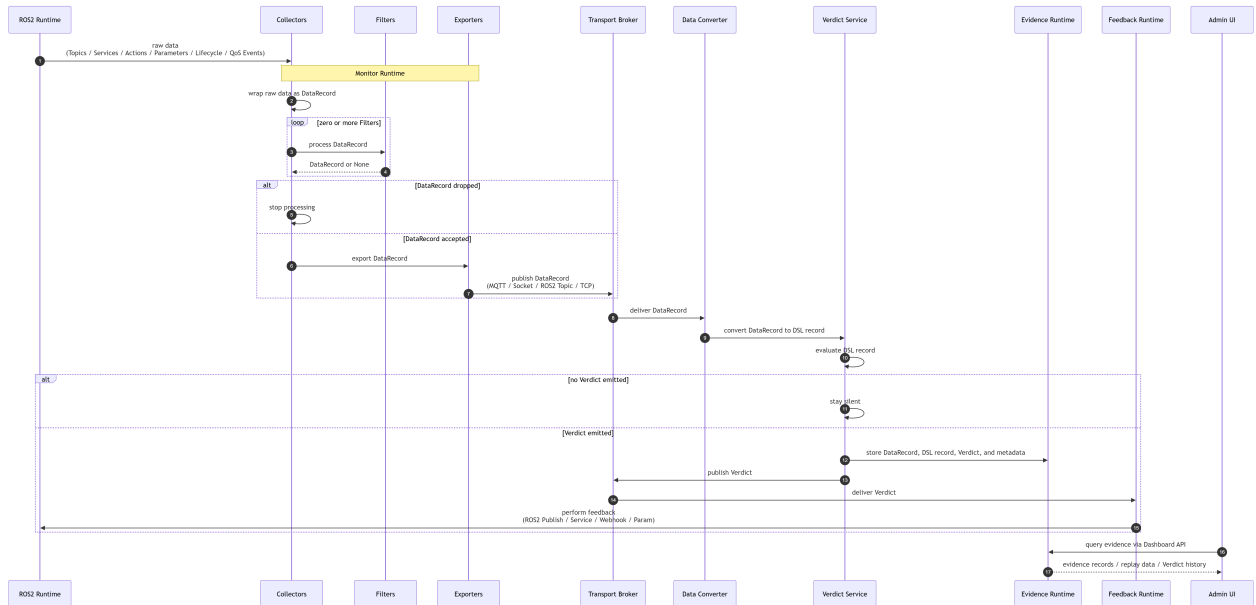


Figure 4.5: Runtime sequence from collection to verdict export.

Chapter 5

Implementation

This chapter describes the Python prototype that implements the reference architecture from Chapter 4. The prototype follows the Pipes and Filters pattern: data moves through small processing stages, and each stage performs one task before passing the result on [27]. This structure matches the pipeline roles introduced in Figure 4.2: collectors create data records, transformers process records, dispatchers route them, converters prepare data for checking, and verdict services produce verdicts.

The chapter first gives the technology choices and the code organization. It then follows the runtime path of a record from collection to transformation, export, checking, and verdict output. The final sections describe YAML configuration, the verifier-side runtime, and generated deployment configurations.

5.1 | Technology Choices and Code Organization

The prototype is implemented in Python. ROS 2 provides the Python client library `rclpy`, which allows the monitor to join the ROS graph as a normal node. Python also has runtime type utilities through `rosidl_runtime_py`. These utilities resolve ROS 2 message, service, and action classes from their type names, so a configured source can be collected without generating source-specific monitoring code. Python also keeps project-specific converters and verdict services simple to add.

Only the collector layer imports ROS 2 packages. The remaining modules use the Python standard library, `PyYAML` for configuration parsing, and `paho-mqtt` for MQTT transport. As a result, the verifier side of a split deployment can run on a plain Python installation. If `paho-mqtt` is unavailable, MQTT components are skipped at construction time while the rest of the modules remain importable and testable.

The repository separates infrastructure code from project-specific checking logic. The `monitor/` package contains the reusable monitoring infrastructure. The `custom/` package contains converters and verdict services used by the case studies. Table 5.1 maps the main modules in `monitor/` to the pipeline roles from Chapter 4.

Module	Responsibility
<code>monitor_node.py</code>	ROS 2-facing entry point for integrated and robot-side monitoring.
<code>node_runner.py</code>	ROS-free entry point for verifier-side processes.
<code>collector.py</code>	Topic, service, and action collectors.
<code>data_record.py</code>	The common data record and its JSON serialization.
<code>transformer.py</code>	Transformer pipeline and built-in transformers.
<code>exporter.py</code> , <code>exporters.py</code> , <code>exporter_mqtt.py</code>	Exporter interface, dispatcher, file exporter, and MQTT exporter.
<code>source.py</code> , <code>sources.py</code> , <code>source_file.py</code> , <code>source_mqtt.py</code>	Verifier-side source interface, file replay source, and MQTT source.
<code>converter.py</code> , <code>verdict.py</code> , <code>verdict_exporters.py</code>	Converter and verdict-service base classes, verdict records, and verdict exporters.
<code>dsl_transport.py</code>	File and MQTT transports for converter output.
<code>config_model.py</code> , <code>pipeline.py</code> , <code>runtime_builder.py</code>	Typed configuration model and builders for runtime wiring.
<code>config_gen.py</code>	Generator that turns one deployment description into runtime configuration files.

Table 5.1: Main modules of the `monitor/` package and their pipeline responsibilities.

5.2 | The Monitor Process

The monitor process is the ROS 2-facing runtime. It is used in the integrated deployment and on the robot side of the split deployment. It is implemented in `monitor_node.py` as one ROS 2 node.

At startup, the process loads a YAML file into the typed configuration model. It creates a session identifier from the current time and a short random component, then builds the configured data exporters and the checking graph. The process waits briefly for DDS discovery and reads the visible topic and service names from the ROS graph. This discovered type information is used only as a fallback. If a source type is already given in the configuration, that configured type is used.

For each configured topic, service, or action, the process creates one collector and one transformer pipeline. A source can also define its own exporters. In that case, records from that source go to a source-specific dispatcher. This is useful when, for example, one high-rate topic should be sent to MQTT while other records are written to a file. All records are also passed to the checking graph when one is configured.

Each run is marked with two session records. The first record is `session_start`; it contains the complete parsed configuration, so an output file can be interpreted later without the original YAML file. On shutdown, collectors are stopped, a `session_end` record with summary statistics is emitted, and exporters are flushed and closed.

5.3 | Collecting ROS 2 Observations

Collectors implement the observation mechanisms described in Section 2.3. They share a base class that owns the per-source sequence counter, the transformer pipeline, and the export callback. The concrete collectors differ in the ROS 2 entity they subscribe to.

Topics. The topic collector resolves the configured message type to a Python class at runtime. It then creates a normal ROS 2 subscription with a configurable QoS depth. Each received message is converted to a dictionary with `message_to_ordereddict`. The same collector code can therefore handle any topic type that is available in the runtime environment.

Services. The service collector subscribes to the event topic of the configured service. Its name is formed by appending `/_service_event` to the service name. This topic exists when service introspection is enabled on the service client or server, as explained in Section 2.3.2. The collector derives the event message class from the configured service type and subscribes with a reliable, transient-local QoS profile. Each event is converted into a request or response record. The event type determines the phase, the request or response payload becomes the record data, and the client identifier and sequence number are kept in the metadata for request-response correlation.

Actions. The action collector subscribes to the hidden feedback and status topics of the configured action: `<action>/_action/feedback` and `<action>/_action/status`. The feedback type is resolved from the action type. The status topic uses the standard goal-status array message from `action_msgs`. The configured phases decide whether feedback, status, or both are collected. Goal, result, and cancellation exchanges use services inside ROS 2 actions. The prototype leaves those phases to service introspection and does not collect them in the action collector.

Collectors also try to record the source node that produced an observation. DDS discovery may still be incomplete when the first message arrives. In that case, ROS 2 can report a placeholder publisher name. The collectors do not cache such placeholders. They retry the lookup on later callbacks and store the node name once a real name is available. Until then, the record simply omits the source-node field.

5.4 | Records and Transformers

The common data record described in Table 4.2 is implemented as a Python dataclass. It is the data unit that moves through the pipeline. Each record stores the record kind, session identifier, source type, source name, message type, timestamp, payload, and optional metadata.

Records are serialized as JSON Lines (JSONL), with one JSON object per line. Empty optional fields are omitted. Deserialization restores the same record fields, so records can pass through files or network transports without losing their monitoring context. A data record identifier combines the session identifier, source type, source name, phase, and per-collector sequence number. The identifier is unique within one session and remains readable in logs and output files. The following shortened JSONL line shows a topic record after field extraction:

```
{ "_type": "data", "session_id": "20260709_142401_5f2c31ab",
  "record_id": "20260709_142401_5f2c31ab:topic:/odom:-:17",
  "source_type": "topic", "source_name": "/odom",
  "msg_type": "nav_msgs/msg/Odometry", "timestamp": 1783002941.312,
  "data": {"twist.twist.linear.x": 0.62},
  "metadata": {"seq": 17, "transformers_applied":
    ["FieldExtractor", "RateThrottler"]}}
```

Each collector sends its records through a transformer pipeline before they are exported or checked. Transformers run in the order given in the configuration. A transformer can return a modified record, the same record, or no record. Returning no record drops the observation and stops the remaining transformers for that observation.

Three transformers are built in:

- **FieldExtractor** keeps selected fields addressed with dot notation. For example, it can keep the linear velocity field of an odometry message. It also flattens selected nested fields into flat keys.
- **RateThrottler** limits the maximum record rate for one source with a monotonic clock. Records above the configured rate are dropped.
- **OnChangeFilter** keeps a record only when at least one watched field changed since the previous record that passed the filter.

The names of the transformers that processed a record are appended to the metadata. Downstream components can therefore see which reductions were applied before storage, transport, or checking.

5.5 | Exporters, Sources, and Transports

An exporter is a sink for records. It accepts a record and can later be flushed or closed. A dispatcher is an exporter that forwards each record to several other exporters. If one exporter fails, the dispatcher reports the failure and continues with the remaining exporters. The same dispatcher implementation is used for data records, converter output, and verdicts.

The prototype includes file and MQTT data exporters. The file exporter appends JSONL records to a per-session file and flushes at a configurable interval. The MQTT exporter publishes data records to topics derived from the source: `<prefix><source_type>/<source_name>`. Session records are published on a separate `<prefix>_session` topic. The connection is established asynchronously, reconnects automatically, and uses a bounded outbound queue so that a slow broker does not block ROS 2 callback threads.

A source is the verifier-side counterpart of an exporter. It reads records from a transport and injects them into a dispatcher. The MQTT source subscribes to a topic filter and parses each payload into a data record. The file source replays a JSONL file. With the `follow` option, it keeps reading new lines appended to the file. Without `follow`, it replays the file once, optionally with a delay between records.

Converter output can also cross a transport boundary. The same file and MQTT carriers are implemented for converter output, and verdicts have file, standard-output, and MQTT exporters. Because each boundary is represented by an exporter on one side and a source on the other, a pipeline stage can run in-process or across a transport without changing the stage implementation.

5.6 | Converters and Verdict Services

Checking is split into two component types. A converter receives a data record. It can return a checker-ready dictionary, a derived data record for another converter, or no output. A converter may also run its own timer, which is needed for deadline properties. A verdict service evaluates checker-ready data and may return a verdict. A verdict contains a timestamp, a property identifier, a boolean result, and details.

The infrastructure attaches correlation data to each verdict. This includes the monitor session and the identifiers of the records that led to the verdict. A later analysis can therefore connect a verdict to the observations behind it. Converters and verdict services do not import ROS 2 packages and do not assume a particular property language.

The concrete checking components used in the case studies live in the `custom/` package. They are selected from YAML by import path:

- **RuleBasedConverter** maps configured record fields to a flat dictionary for records whose source matches a configured pattern. It is used for threshold-style checks.
- **ThresholdVerdict** compares one field with a bound. It emits a violation verdict when the violation starts, and a recovery verdict when the value returns inside the bound. It can require the violation to last for a configured duration before reporting it.
- **ResetPoseEffectConverter** checks a property over two interfaces. After a successful `/reset_pose` service response, the odometry should return close to the origin within a deadline. The converter correlates service records with odometry records and uses a timer to report a violation when the deadline expires.
- **RelativeSpeedConverter** joins odometry streams from two robots and emits their relative speed. This demonstrates a property that uses more than one data source.

5.7 | Configuration and Graph Wiring

One YAML file configures one runtime. The configuration is parsed into typed specification objects. Built-in components are selected by short names such as `file` or `mqtt`. Project-specific components are selected by an import path of the form `module.path:ClassName`. When a class is resolved, the builder checks that it implements the expected base class before constructing it. Extra keys in a plugin entry are passed to the constructor as keyword arguments, so built-in and project-specific components use the same configuration style.

The following excerpt observes one odometry topic, extracts the linear speed, exports data records, and checks a speed limit:

```
topics:
  - name: /odom
    type: nav_msgs/msg/Odometry
    transformers:
      - {type: FieldExtractor, fields: [twist.twist.linear.x]}
      - {type: RateThrottler, max_rate_hz: 5.0}

exporters:
  - {type: file}
  - {type: mqtt, broker: localhost, topic_prefix: monitor/, qos: 1}

converters:
  - id: odom_speed
    type: custom.rule_based:RuleBasedConverter
    params: {source_match: "~/odom$",
             field_map: {velocity: twist.twist.linear.x}}

verdict_services:
  - id: odom_speed_check
    type: custom.threshold:ThresholdVerdict
    params: {property_id: odom_speed_limit, field: velocity,
             op: ">", threshold: 0.5}
    exporters:
      - {type: file, path: verdicts_{session_id}.jsonl}

links:
  - {from: "source:/odom", to: "converter:odom_speed"}
```

```
- {from: "converter:odom_speed", to: "verdict:odom_speed_check"}
```

The **links** section defines the checking graph. Nodes are addressed as **source:<name>**, **converter:<id>**, **verdict:<id>**, **input:<id>**, and **output:<id>**. Edges connect record streams to converters, converters to further converters or verdict services, and transport endpoints to graph nodes.

The graph builder turns this description into live objects. It creates each component once, wraps converters with source filters when needed, builds converter chains in downstream-first order, and rejects cyclic links. The monitor process and the verifier runtime use the same graph builder, which keeps component placement in the configuration files.

5.8 | The Verifier Runtime

The verifier-side entry point is **node_runner.py**. It runs the same checking graph without a ROS 2 dependency. Its configuration uses **inputs** in place of collector sections. An input receives data records or converter output from MQTT or a file and feeds them into the graph.

A single runner process can host converters, verdict services, or both. It can therefore act as a complete verifier, one half of a separated checking chain, or a relay that filters records and republishes them. The runtime role is determined by the YAML file.

The integrated deployment runs the full pipeline in one monitor process. The split deployment runs the monitor process near the robot and sends serialized records to one or more verifier runtimes. Since both deployments exchange the same data record format, a recorded JSONL file can later be replayed through the same checking graph for offline analysis.

5.9 | Generating Deployment Configurations

A multi-host deployment needs several YAML files that agree on transports, topic names, and payload kinds. Writing these files by hand is easy to get wrong. The prototype therefore includes a configuration generator that derives the runtime YAML files from one deployment description.

The deployment description is a JSON document. The **hosts** section lists machines. Each host contains runtimes. A **ros2** runtime declares the observable sources offered by an application. A **monitor** runtime subscribes to selected sources. **converter** and **verdict_service** runtimes define checking components and their parameters. The **links** section connects runtimes on different hosts and states whether the link carries data records or converter output, and whether it uses MQTT or a shared file.

The request model is intentionally close to the runtime concepts. A **Host** is a placement boundary. A **Runtime** is one executable role placed on a host. A **Source** is a topic, service, or action declared by a **ros2** runtime. A **Link** connects two runtimes and contains a **Transport**. Existing transport kinds are **mqtt** and **file**. Existing link payloads are **records**, for data records, and **dsl**, for converter output.

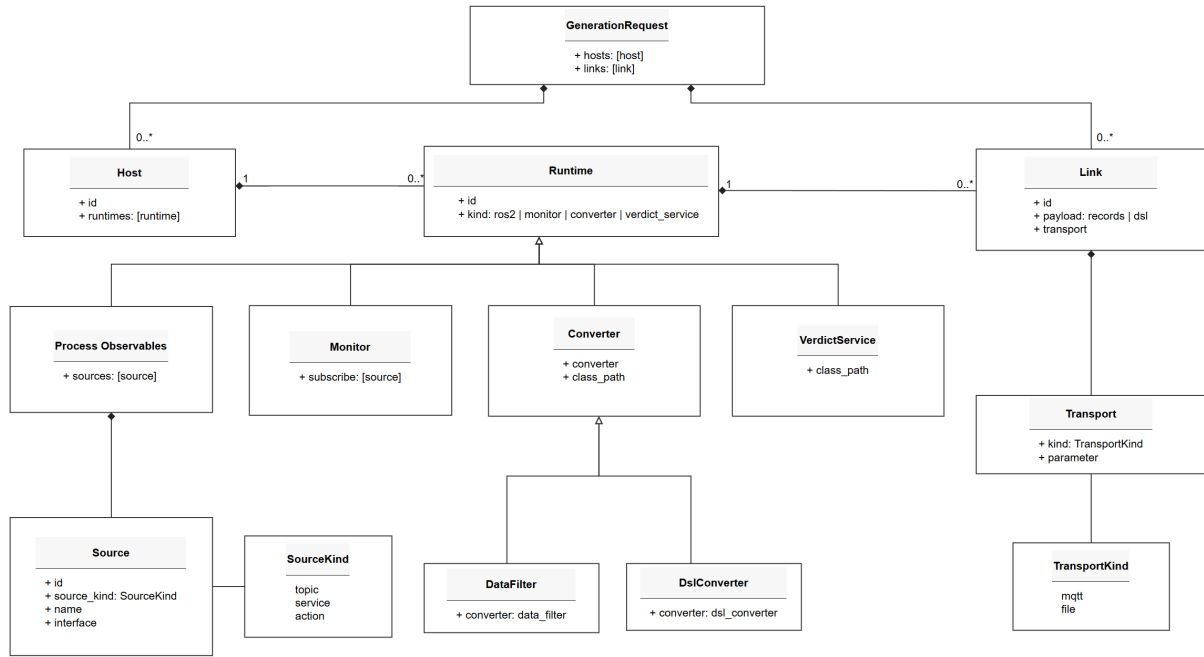


Figure 5.1: Data model used by the configuration generator. A generation request contains hosts and links. Hosts contain runtimes, process observables declare monitorable sources, and links define the transport used between runtimes.

The following example places a monitor next to the robot and the checking chain on a separate verifier host:

```

{"hosts": [
  {"id": "robot", "runtimes": [
    {"id": "app", "kind": "ros2", "sources": [
      {"id": "odom", "source_kind": "topic", "name": "/odom",
        "interface": "nav_msgs/msg/Odometry"}]},
    {"id": "mon", "kind": "monitor", "subscribe": ["odom"]}],
  {"id": "verifier", "runtimes": [
    {"id": "conv", "kind": "converter",
      "class_path": "custom.rule_based:RuleBasedConverter",
      "input_from": ["odom"], "output_to": "speed", ...},
    {"id": "check", "kind": "verdict_service",
      "class_path": "custom.threshold:ThresholdVerdict",
      "input_from": ["speed"], ...}]},
  "links": [
    {"from_host": "robot", "to_host": "verifier",
      "from_runtime": "mon", "to_runtime": "conv",
      "payload": "records",
      "transport": {"kind": "mqtt", "broker": "192.168.1.10"}}]}
  
```

Algorithm 1 shows the projection used by the generator. The algorithm first builds an index of sources, runtimes, host placement, and cross-host links. It then emits at most one YAML file per host: a monitor configuration for a host with a monitor runtime, or a runner configuration for a host that only runs checking components.

The generator projects this description onto one YAML file per host. A host with a monitor runtime receives a monitor-process configuration. A host with only converters or verdict services receives a runner configuration. A host that only runs the observed ROS 2 application does not need a generated file.

Algorithm 1 Project a deployment request into runtime YAML files

Require: deployment request J **Ensure:** generated configurations F

```

1:  $F \leftarrow \emptyset$ 
2:  $I \leftarrow \text{BUILDINDEX}(J.\text{hosts}, J.\text{links})$ 
3: for all host  $H$  in  $J.\text{hosts}$  do
4:    $K \leftarrow$  the set of runtime kinds on  $H$ 
5:   if monitor is in  $K$  then
6:      $Y \leftarrow \text{NEWMONITORCONFIG}(H)$ 
7:     for all monitor runtime  $M$  on  $H$  do
8:        $E \leftarrow \text{RECORDEXPORTERS}(M, I)$ 
9:       for all source id  $s$  in  $M.\text{subscribe}$  do
10:        add  $\text{SOURCEENTRY}(I.\text{sources}[s], E)$  to  $Y$ 
11:      end for
12:    end for
13:     $G \leftarrow \text{NEWGRAPHSECTION}(H, I)$ 
14:    for all converter runtime  $C$  on  $H$  do
15:      reject unsupported cross-host record inputs to  $C$ 
16:       $\text{ADDCONVERTER}(G, C)$ 
17:    end for
18:    for all verdict-service runtime  $V$  on  $H$  do
19:      reject unsupported cross-host DSL inputs to  $V$ 
20:       $\text{ADDVERDICT}(G, V)$ 
21:    end for
22:     $\text{APPLYGRAPH}(G, Y)$ 
23:     $F[H.\text{id}] \leftarrow Y$ 
24:  else if  $K$  contains converter or verdict_service then
25:     $Y \leftarrow \text{NEWRUNNERCONFIG}(H)$ 
26:     $G \leftarrow \text{NEWGRAPHSECTION}(H, I)$ 
27:    for all converter runtime  $C$  on  $H$  do
28:      add incoming cross-host record inputs for  $C$ 
29:       $\text{ADDCONVERTER}(G, C)$ 
30:    end for
31:    for all verdict-service runtime  $V$  on  $H$  do
32:      add incoming cross-host DSL inputs for  $V$ 
33:       $\text{ADDVERDICT}(G, V)$ 
34:    end for
35:     $\text{APPLYGRAPH}(G, Y)$ 
36:     $F[H.\text{id}] \leftarrow Y$ 
37:  end if
38: end for
39: return  $F$ 

```

Runtimes on the same host are wired in-process by matching their declared record identifiers. Cross-host communication requires an explicit link. The writer and reader of a link derive their transport namespace from the same link identity, so they use matching MQTT topics or shared file paths. Unsupported placements are rejected during generation. For example, a converter on a monitor host cannot consume a cross-host record stream, because cross-host record inputs are handled by the runner runtime.

For repeatable deployment, the repository also provides a container image based on a ROS 2 base image. The image packages the monitor, the custom components, and their dependencies. A Docker Compose service runs the monitor on the host network so that DDS discovery can reach the robot nodes.

Chapter 6

Evaluation

Chapter 7

Discussion

Chapter 8

Conclusions

Bibliography

- [1] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022. 4, 8, 10, 13
- [2] Open Robotics. Topics vs services vs actions. ROS 2 Documentation. Accessed 2026-06-06. 4, 8, 10
- [3] Rick Rabiser, Klaus Schmid, Holger Eichelberger, Michael Vierhauser, Sam Guinea, and Paul Grünbacher. A domain analysis of resource and requirements monitoring: Towards a comprehensive model of the software monitoring domain. *Information and Software Technology*, 111:86–109, 2019. 4, 7, 11, 13, 19
- [4] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009. 4, 11
- [5] Angelo Ferrando, Rafael C. Cardoso, Michael Fisher, Davide Ancona, Luca Franceschini, and Viviana Mascardi. ROSMonitoring: A runtime verification framework for ROS. In *Towards Autonomous Robotic Systems*, volume 12228 of *Lecture Notes in Computer Science*, pages 387–399. Springer, 2020. 7, 13, 19
- [6] Marco Stadler and Michael Vierhauser. ROMoSu: Flexible runtime monitoring support for ROS-based applications. In *2023 IEEE/ACM 5th International Workshop on Robotics Software Engineering*, pages 53–60, 2023. 7, 20
- [7] Joakim Schack Betzer, Jalil Boudjadar, Mirgita Frasheri, and Prasad Talasila. Digital twin enabled runtime verification for autonomous mobile robots under uncertainty, 2024. 7, 21
- [8] Open Robotics. Ros 2 topics. ROS 2 Documentation. Accessed 2026-06-06. 8
- [9] Geoffrey Biggs, Jacob Perron, and Shane Loretz. Actions. ROS 2 Design, 2019. Accessed 2026-06-06. 10
- [10] Open Robotics. Quality of service settings. ROS 2 Documentation. Accessed 2026-06-06. 10
- [11] Ivan Kurtev and Jozef Hooman. Runtime verification of compound components with ComMA. In *A Journey from Process Algebra via Timed Automata to Model Learning*, volume 13560 of *Lecture Notes in Computer Science*, pages 382–402. Springer, 2022. 11
- [12] Adrian Francalanza, Jorge A. Pérez, and César Sánchez. Runtime verification for decentralised and distributed systems. In Ezio Bartocci and Yliès Falcone, editors, *Lectures on Runtime Verification: Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 176–210. Springer, 2018. 12, 15, 16, 17, 21

- [13] Open Robotics. Configure service introspection. ROS 2 Documentation. Accessed 2026-06-06. 12
- [14] Maryam Ghaffari Saadat, Angelo Ferrando, Louise A. Dennis, and Michael Fisher. ROS-Monitoring 2.0: Extending ROS runtime verification to services and ordered topics. In *Proceedings of the Sixth International Workshop on Formal Methods for Autonomous Systems*, volume 411 of *Electronic Proceedings in Theoretical Computer Science*, pages 38–55, 2024. 13, 19
- [15] Sean Rivera, Antonio Ken Iannillo, Sofiane Lagraa, Clément Joly, and Radu State. ROS-FM: Fast monitoring for the robotic operating system (ROS). In *2020 25th International Conference on Engineering of Complex Computer Systems*, pages 15–24, 2020. 13, 21
- [16] Christophe Bédard, Ingo Lütkebohle, and Michel R. Dagenais. `ros2.tracing`: Multipurpose low-overhead framework for real-time tracing of ROS 2. *IEEE Robotics and Automation Letters*, 7(3):6511–6518, 2022. 13, 20
- [17] Ivan Perez, Anastasia Mavridou, Tom Pressburger, Alexander Will, and Patrick J. Martin. Monitoring ROS2: From requirements to autonomous robots. In *Proceedings of the Fourth Workshop on Formal Methods for Autonomous Systems*, volume 371 of *Electronic Proceedings in Theoretical Computer Science*, pages 208–216, 2022. 13, 20
- [18] Ricardo Caldas, Juan Antonio Piñera García, Matei Schiopu, Patrizio Pelliccione, Genáina Rodrigues, and Thorsten Berger. Runtime verification and field-based testing for ROS-based robotic systems. *IEEE Transactions on Software Engineering*, 50(10):2544–2567, 2024. 13, 21
- [19] Kyo C. Kang, Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. Feature-oriented domain analysis (FODA) feasibility study. Technical Report CMU/SEI-90-TR-021, Software Engineering Institute, Carnegie Mellon University, 1990. 14
- [20] David Benavides, Sergio Segura, and Antonio Ruiz-Cortés. Automated analysis of feature models 20 years later: A literature review. *Information Systems*, 35(6):615–636, 2010. 14
- [21] OASIS MQTT Technical Committee. MQTT version 5.0. OASIS Standard, 2019. Accessed 2026-06-25. 18
- [22] Marco Stadler, Michael Vierhauser, and Jane Cleland-Huang. Towards flexible runtime monitoring support for ROS-based applications. In *Proceedings of the 4th International Workshop on Robotics Software Engineering*, pages 12–15. ACM, 2022. 20
- [23] Tomoya Yamaguchi, Bardh Hoxha, and Dejan Ničković. RTAMT: Runtime robustness monitors with application to CPS and robotics. *International Journal on Software Tools for Technology Transfer*, 26(1):79–99, 2024. 20
- [24] Elif Değirmenci, Yunus Sabri Kırca, İlker Özçelik, and Ahmet Yazıcı. ROSIDS23: Network intrusion detection dataset for robot operating system. *Data in Brief*, 51:109739, 2023. 21
- [25] Tommaso Puccetti, Simone Nardi, Cosimo Cinquilli, Tommaso Zoppi, and Andrea Ceccarelli. ROSPaCe: Intrusion detection dataset for a ROS2-based cyber-physical system and IoT networks. *Scientific Data*, 11(1):481, 2024. 21

- [26] Stefano Aldegheri, Nicola Bombieri, Samuele Germiniani, Federico Moschin, and Graziano Pravadelli. A containerized ROS-compliant verification environment for robotic systems. In *2021 Design, Automation and Test in Europe Conference and Exhibition*, pages 222–225, 2021. 21
- [27] Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, and Michael Stal. *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. John Wiley & Sons, 1996. 32

Appendix A

Appendix